SET 13

PART A — Short questions and answers

Q1. Explain the role of the relationship attribute Fee_Paid in the Participant–Event registration.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q2. Summarize the effect of an UPDATE command that has no WHERE clause.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q3. Compare 2NF and 3NF.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q4. Explain how IS NULL differs from = NULL when searching for unpaid registrations.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Describe the result of an aggregate function used without GROUP BY.

Answer: GROUP BY collects rows with the same value so aggregate functions can calculate a result for each group.

Q6. Describe how a stored procedure is invoked from an anonymous PL/SQL block.

Answer: A stored procedure is a named PL/SQL program saved in the database. It can accept parameters and perform reusable operations.

Q7. Explain the term NoSQL.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q8. Describe the data types commonly stored in a document, such as string, number, date and array.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q9. Interpret the MongoDB query: db.events.find({EntryFee: {$lte: 100}})

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Interpret the Cypher query: MATCH (p:Player) RETURN count(p);

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: An inter-college sports meet maintains a Sports Event Registration System to store participant details,

1(a) ER design:

• **PARTICIPANT**(ParticipantID, ParticipantName, College, Gender); primary key: **ParticipantID**.

• **EVENT**(EventID, EventName, Venue, EventDate, EntryFee); primary key: **EventID**.

• **REGISTRATION**(RegID, ParticipantID, EventID, RegDate, FeePaid); primary key: **RegID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE PARTICIPANT (
  ParticipantID NUMBER PRIMARY KEY,
  ParticipantName VARCHAR2(100),
  College VARCHAR2(100),
  Gender VARCHAR2(100)
);
```

```
CREATE TABLE EVENT (
  EventID NUMBER PRIMARY KEY,
  EventName VARCHAR2(100),
  Venue VARCHAR2(100),
  EventDate DATE,
  EntryFee VARCHAR2(100)
);
CREATE TABLE REGISTRATION (
  RegID NUMBER PRIMARY KEY,
  ParticipantID NUMBER,
  EventID NUMBER,
  RegDate DATE,
  FeePaid NUMBER
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM REGISTRATION x
JOIN PARTICIPANT a ON x.ParticipantID = a.ParticipantID
JOIN EVENT b ON x.EventID = b.EventID;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. MongoDB: Create the collections and insert documents using the fields listed in the set. Query with find() and the requested filter.

```
db.collection.insertMany([
  { _id: 1, name: 'Sample A', category: 'Example', price: 500 },
  { _id: 2, name: 'Sample B', category: 'Example', price: 300 }
]);
db.collection.find({ author: 'Specified Author' });
db.collection.find({ price: { $gt: 400 } });
```